

QMM Complete Workflow Manual

Diego Juarez and Codex

June 25, 2026

Contents

1	Purpose	3
2	QMM Organization	3
3	What The JSON File Controls	4
3.1	Documentation Blocks Inside The Same JSON	4
3.2	What The JSON Cannot Do By Itself	5
4	Units and Default Physical Inputs	5
5	Interaction Models	5
5.1	Attractive Mean Fields	6
5.2	Excluded-Volume Prescriptions	6
5.2.1	vdW / Standard Excluded Volume	6
5.2.2	Carnahan–Starling Excluded Volume	7
5.2.3	TVM Excluded Volume	7
5.2.4	Species-Level Excluded Volume in Asymmetric Matter	7
5.3	Examples Already Implemented	7
6	Workflow 1: Symmetric Zero-Temperature Ground State	8
7	Workflow 2: Finite-Temperature Liquid Critical Point	8
8	Workflow 3: Symmetric Quarkyonic Matter	9
8.1	Energy Minimization, Golden Section, and Momentum Modes	9
8.2	Sound Speed	10
9	Workflow 3A: Pure Hadronic EOS Table	10
10	Workflow 4: Asymmetric Hadronic Matter	11
11	Workflow 5: Asymmetric Quarkyonic Matter	11
12	Workflow 6: Neutron Stars	11

13 How To Read The Workflow Flags	12
13.1 Minimal Symmetric Calibration	12
13.2 Add The Liquid Critical Point	12
13.3 Add Symmetric Quarkyonic Matter	12
13.4 Add A Pure Hadronic EOS Table	12
13.5 Add Asymmetric Matter Without Leptons	12
13.6 Add Leptons	12
13.7 Add Neutron Stars	12
14 How To Add A New Model	13
15 Output Controls	13
16 Practical Limitation	13
17 Main Files To Use	14

1 Purpose

This document explains the workflow implemented in **QMM**. The goal is to keep the physics, numerical methods, code organization, and JSON interface in one clean place.

The package is designed to support two use cases:

- production runs for the models already implemented in `src/qmm/models.py`
- extension to a new real-gas model with a new mean field and/or a new excluded-volume prescription

2 QMM Organization

The main directories are:

- `src/qmm/`: source code
- `examples/`: JSON inputs, including a blank generalized template
- `notebooks/QMM_workflow.ipynb`: single overview notebook
- `docs/qmm_complete_workflow_manual.tex`: this manual
- `results/`: local reference outputs used by the notebook

The main solver modules are:

- `models.py`: interaction models and repulsive mappings
- `fermi_gas.py`: relativistic ideal Fermi gas
- `ground_state.py`: symmetric zero-temperature fit
- `quantum.py`: finite-temperature liquid critical point
- `quarkyonic.py`: symmetric and asymmetric quarkyonic matter
- `sound_speed.py`: sound-speed reconstruction
- `asymmetry.py`: asymmetric hadronic couplings
- `neutron_star.py`: beta-equilibrium EOS preparation and TOV solver
- `runner.py`: top-level workflow orchestration
- `config.py`: JSON loading and typed configuration

3 What The JSON File Controls

The workflow is driven by a single JSON input file. The blank template is:

`examples/qmm_blank_template.json`

The richer single-file design template is:

`examples/qmm_all_in_one_template.json`

The JSON file can decide:

- which registered model is used
- whether an extra model parameter is fixed or solved from a target incompressibility
- whether to compute the symmetric ground-state point
- whether to compute the liquid critical point
- whether to compute a dense pure-hadronic EOS table
- whether to compute symmetric quarkyonic matter
- whether to compute asymmetric couplings
- whether to compute asymmetric fixed- y quarkyonic matter
- whether to include leptons through beta equilibrium
- whether to compute neutron stars
- whether CSV, JSON, and plot outputs are written

3.1 Documentation Blocks Inside The Same JSON

The all-in-one template also contains descriptive blocks such as:

- `documentation.mean_field_latex`
- `documentation.excluded_volume_latex`
- `documentation.ideal_density_map_latex`
- `documentation.pressure_prefactor_latex`
- `requested_calculations.*`

These fields are intentionally not executable physics inputs. They are there so one JSON file can also serve as a human-readable model record. The QMM loader ignores these extra descriptive keys.

3.2 What The JSON Cannot Do By Itself

The JSON file does not encode new analytic physics functions. In particular, it cannot by itself define:

- a new mean field $U(n)$
- a new derivative dU/dn
- a new density mapping $n \leftrightarrow n_{\text{id}}$
- a new excluded-volume fraction
- a new pressure prefactor

Those must be implemented once in `src/qmm/models.py`. After the model is registered there, the same JSON interface can drive it.

4 Units and Default Physical Inputs

The code uses MeV–fm units for the nuclear-matter solvers and converts to cgs only when solving the TOV equations.

The default physical constants are:

$$N_c = 3 \quad \text{color,} \tag{1}$$

$$g = 4 \quad \text{for symmetric nuclear matter,} \tag{2}$$

$$g = 2 \quad \text{for each nucleon species in asymmetric matter,} \tag{3}$$

$$\hbar c = 197.3269804 \text{ MeV fm,} \tag{4}$$

$$m_p = 938.27208816 \text{ MeV,} \tag{5}$$

$$m_N = 938.0 \text{ MeV,} \tag{6}$$

$$m_u^{\overline{\text{MS}}}(2 \text{ GeV}) = 2.16 \text{ MeV,} \tag{7}$$

$$m_d^{\overline{\text{MS}}}(2 \text{ GeV}) = 4.67 \text{ MeV,} \tag{8}$$

$$n_0 = 0.16 \text{ fm}^{-3}, \tag{9}$$

$$E/A - m_N = -16 \text{ MeV at saturation,} \tag{10}$$

$$J = 32.5 \text{ MeV,} \tag{11}$$

$$L = 58.9 \text{ MeV.} \tag{12}$$

5 Interaction Models

Each model is represented by one `InteractionModel` entry in `models.py`. The definition requires the following ingredients:

- attractive function $U(n, b, \text{parameter})$
- derivative dU/dn
- map $n \mapsto n_{\text{id}}$

- inverse map $n_{\text{id}} \mapsto n$
- total volume fraction
- species-level volume fraction
- pressure prefactor
- capability flags for each workflow branch

5.1 Attractive Mean Fields

The attractive sectors implemented in **QMM** are:

$$U_{\text{vdW}}(n) = -n, \quad (13)$$

$$U_{\text{RKS}}(n) = -\frac{1}{b} \ln(1 + bn), \quad (14)$$

$$U_{\text{PR}}(n) = -\frac{1}{2\sqrt{2}b} \ln \left[\frac{1 + (1 + \sqrt{2})bn}{1 + (1 - \sqrt{2})bn} \right], \quad (15)$$

$$U_{\text{Clausius}}(n) = -\frac{n}{1 + cn}, \quad (16)$$

$$U_{\text{Dieterici}}(n) = -\frac{n^{\alpha-1}}{\alpha - 1}. \quad (17)$$

For the **cs** and **tvm** models, the attractive sector remains the van der Waals form $U(n) = -n$, while the repulsive sector is modified through the excluded-volume mapping. The hybrid models **clausius_cs** and **clausius_tvm** keep the Clausius attractive sector and only replace the repulsive mapping.

5.2 Excluded-Volume Prescriptions

The repulsive sector in **QMM** is encoded through:

- a volume fraction $f(n, b)$
- an ideal-density map $n \mapsto n_{\text{id}}$
- a pressure prefactor $\Pi(n, b)$

5.2.1 vdW / Standard Excluded Volume

The standard excluded-volume prescription used by **vdw**, **rks**, **pr**, **clausius**, and **dieterici** is:

$$f_{\text{EV}}(n) = 1 - bn, \quad (18)$$

$$n_{\text{id}} = \frac{n}{1 - bn}, \quad (19)$$

$$\Pi_{\text{EV}}(n, b) = 1. \quad (20)$$

5.2.2 Carnahan–Starling Excluded Volume

For the `cs` model, and also for the hybrid `clausius_cs`, define $x = bn$. The excluded-volume fraction is:

$$f_{\text{CS}}(x) = \exp \left[-\frac{3x}{4-x} - \frac{4x}{(4-x)^2} \right], \quad (21)$$

$$n_{\text{id}} = \frac{n}{f_{\text{CS}}(x)}. \quad (22)$$

The pressure prefactor is:

$$\Pi_{\text{CS}}(n, b) = f_{\text{CS}}(n, b) - n \frac{df_{\text{CS}}(n, b)}{dn}. \quad (23)$$

5.2.3 TVM Excluded Volume

For the `tvm` model, and also for the hybrid `clausius_tvm` model, define again $x = bn$. The TVM excluded-volume fraction is:

$$f_{\text{TVM}}(x) = \exp \left(-x - \frac{x^2}{2} \right), \quad (24)$$

$$n_{\text{id}} = \frac{n}{f_{\text{TVM}}(x)}. \quad (25)$$

The pressure prefactor is:

$$\Pi_{\text{TVM}}(n, b) = f_{\text{TVM}}(n, b) - n \frac{df_{\text{TVM}}(n, b)}{dn}. \quad (26)$$

5.2.4 Species-Level Excluded Volume in Asymmetric Matter

For asymmetric matter, the code uses the species-dependent arguments

$$x_p = b_n n_p + b_{pn} n_n, \quad (27)$$

$$x_n = b_{pn} n_p + b_n n_n, \quad (28)$$

and then evaluates the species volume fraction with the corresponding model rule: $1 - x$ for standard excluded volume, $f_{\text{CS}}(x)$ for Carnahan–Starling, or $f_{\text{TVM}}(x)$ for TVM.

5.3 Examples Already Implemented

The package already contains the following models:

- `vdw`
- `rks`
- `pr`
- `clausius`

- `clausius_cs`
- `clausius_tvm`
- `dieterici`
- `cs`
- `tvm`

The hybrid model `clausius_cs` combines the Clausius attractive sector with the Carnahan–Starling excluded-volume mapping, while `clausius_tvm` combines the same Clausius attraction with the TVM repulsive mapping. Because these are not the original pure Clausius model, the practical search window for the parameter c may differ from the usual pure-Clausius reference range. In the included scan examples, the search interval is widened to reach low- K_0 targets such as $K_0 = 240$ MeV.

6 Workflow 1: Symmetric Zero-Temperature Ground State

At saturation density the code solves for the couplings a and b using:

$$P(n_0) = 0, \quad (29)$$

$$\frac{\epsilon(n_0)}{n_0} - m_N = -16 \text{ MeV}. \quad (30)$$

After solving a and b , the incompressibility is evaluated as

$$K_0 = 9 \left. \frac{dP}{dn_B} \right|_{n_0}. \quad (31)$$

The symmetric energy density and pressure use the model-dependent ideal-density map and mean field:

$$\epsilon(n) = f(n, b) \epsilon_{\text{id}}(n_{\text{id}}) + n a U(n), \quad (32)$$

$$P(n) = \Pi(n, b) P_{\text{id}}(n_{\text{id}}) + n^2 a \frac{dU}{dn}. \quad (33)$$

7 Workflow 2: Finite-Temperature Liquid Critical Point

The finite-temperature critical point is obtained from the interacting pressure surface by solving:

$$\left(\frac{\partial P}{\partial n_B} \right)_T = 0, \quad (34)$$

$$\left(\frac{\partial^2 P}{\partial n_B^2} \right)_T = 0. \quad (35)$$

The code first inverts the finite-temperature ideal-gas density to obtain the effective chemical potential μ^* , then reconstructs the interacting pressure. The outputs are:

- T_c
- n_c
- P_c

8 Workflow 3: Symmetric Quarkyonic Matter

At fixed baryon density n_B , the code minimizes the zero-temperature energy density with respect to the quark fraction f_Q :

$$\epsilon = \epsilon(n_B, f_Q). \quad (36)$$

The quarkyonic shell scale is controlled by the JSON entry `quarkyonic.lambda_momentum_mev`. This is the main place where the CS/TVM versus other-model choices can differ in practice.

8.1 Energy Minimization, Golden Section, and Momentum Modes

The symmetric solver does not assume a fixed quark fraction. Instead, at each chosen density n_B it solves

$$\epsilon_{\min}(n_B) = \min_{f_Q \in [f_{Q,\min}, 1]} \epsilon(n_B, f_Q). \quad (37)$$

Computationally, this is done in two stages:

1. build a coarse scan in f_Q using the JSON value `quarkyonic.fq_scan_points`
2. identify candidate local minima and refine each candidate bracket with a golden-section minimizer until the tolerance `quarkyonic.refine_tol` is reached

The final state is the one with the smallest refined energy density. This is the quantity written later as the raw curve $\epsilon_{\text{raw}}(n_B)$.

The quark integrals use the infrared regulator

$$\kappa = \frac{\lambda}{\hbar c}, \quad \lambda = \text{quarkyonic.lambda_momentum_mev}, \quad (38)$$

and, in the current implementation,

$$n_Q(k) = \frac{gQ}{2\pi^2} \int_0^{k/N_c} q \sqrt{q^2 + \kappa^2} dq, \quad (39)$$

$$\epsilon_Q(k) = \frac{N_c gQ}{2\pi^2} \int_0^{k/N_c} q \sqrt{q^2 + \kappa^2} \sqrt{(\hbar c q)^2 + m_q^2} dq, \quad (40)$$

with $m_q = m_N/N_c$ in the notebook and source-code workflow used here.

Two momentum-space realizations are supported:

- **quarkyonic mode:** quarks occupy the inner Fermi sea up to k_{bu} and baryons fill the outer shell $k_{bu} < k < k_F$
- **baryquark mode:** baryons occupy the inner Fermi sea up to k_F and quarks occupy the shell $k_F < k < k_{bu}$

The new symmetric comparison notebook added in this update uses

- $\lambda = 200$ MeV for the quarkyonic curves
- $\lambda = 0$ for the baryquark curves

so that the baryquark comparison can be inspected without the extra infrared regulator broadening in that branch.

8.2 Sound Speed

Once the minimum-energy curve $\epsilon(n_B)$ has been generated, the code reconstructs:

$$\mu_B = \frac{d\epsilon}{dn_B}, \quad (41)$$

$$P = n_B \mu_B - \epsilon, \quad (42)$$

$$v_s^2 = \frac{dP/dn_B}{\mu_B}. \quad (43)$$

In the current design, if the quarkyonic profile is requested, sound speed is reconstructed automatically as part of the workflow.

For this workflow the code stores both the raw minimum-energy curve $\epsilon_{\text{raw}}(n_B)$ and a locally smoothed curve $\epsilon(n_B)$. The production sound speed is reconstructed from the smoothed curve using a local polynomial regression before taking first and second derivatives. This improves numerical stability of

$$v_s^2 = \frac{n_B d^2\epsilon/dn_B^2}{d\epsilon/dn_B}. \quad (44)$$

For the new baryquark/quarkyonic comparison notebook, the baryquark v_s^2 panel may also be plotted from a direct discrete-gradient diagnostic applied to $\epsilon_{\text{raw}}(n_B)$. That diagnostic is useful when one wants to preserve the narrow peak structure in the visualization, but the smoothed reconstruction remains the default production method written by the main workflow.

9 Workflow 3A: Pure Hadronic EOS Table

The workflow can also write a dense zero-temperature hadronic EOS table before the quarkyonic construction is applied. This is controlled by:

```
workflows.hadronic_eos_table = true
```

The density range and derivative-reconstruction settings are controlled by the `hadronic_eos` block. The output table contains:

- n
- n/n_0
- raw and smoothed $\epsilon(n)$
- $E/A - m_N$
- μ_B
- P
- dP/dn_B
- $d^2\epsilon/dn_B^2$
- v_s^2

This provides a direct hadronic EOS and sound-speed diagnostic without turning on the quarkyonic branch.

10 Workflow 4: Asymmetric Hadronic Matter

The asymmetric hadronic sector is fitted from the five constraints used in this project:

- binding energy
- saturation density
- symmetry energy J
- symmetry-energy slope L
- incompressibility K_0

The asymmetric couplings are:

$$(a_n, a_{pn}, b_n, b_{pn}). \quad (45)$$

The code currently supports two branches:

- `equal_b`: $b_{pn} = b_n$
- `target_l`: $b_{pn} \neq b_n$, chosen to reproduce L

11 Workflow 5: Asymmetric Quarkyonic Matter

Once the asymmetric couplings are known, the code can compute:

- fixed- y profiles for user-selected proton fractions
- beta-equilibrium matter with leptons

The proton-fraction grid is selected in:

`asymmetric.proton_fraction_values`

Leptons are controlled by:

`asymmetric.beta_equilibrium`

If `beta_equilibrium=false`, the code computes only fixed- y matter. If `beta_equilibrium=true`, the code solves for charge neutrality with electrons and muons and constructs the beta-equilibrium EOS.

12 Workflow 6: Neutron Stars

The neutron-star branch uses the beta-equilibrium EOS and integrates the Tolman–Oppenheimer–Volkoff equations with an RK4 solver.

The top-level switch is:

`workflows.neutron_star`

The main numerical controls are:

- number of central-pressure points
- pressure range
- radial step size
- maximum radius

13 How To Read The Workflow Flags

13.1 Minimal Symmetric Calibration

If you only want to solve the symmetric ground-state point and obtain (a, b, K_0) , set:

- `ground_state=true`
- everything else in `workflows` to `false`

13.2 Add The Liquid Critical Point

Set:

- `quantum_critical=true`

13.3 Add Symmetric Quarkyonic Matter

Set:

- `symmetric_quarkyonic=true`

13.4 Add A Pure Hadronic EOS Table

Set:

- `hadronic_eos_table=true`

13.5 Add Asymmetric Matter Without Leptons

Set:

- `asymmetric_fit=true`
- `asymmetric_quarkyonic=true`
- `asymmetric.beta_equilibrium=false`

13.6 Add Leptons

Set:

- `asymmetric_fit=true`
- `asymmetric_quarkyonic=true`
- `asymmetric.beta_equilibrium=true`

13.7 Add Neutron Stars

Set:

- `neutron_star=true`

14 How To Add A New Model

For a new real-gas model, implement the model in `src/qmm/models.py`:

1. define the attractive sector $U(n)$ and dU/dn
2. define the excluded-volume mapping
3. define the inverse density mapping
4. define the volume fraction
5. define the pressure prefactor
6. register a new `InteractionModel`
7. set the capability flags for supported workflow branches

After that, put the registered name into:

`model.name`

inside a JSON file based on either `qmm.blank_template.json` or `qmm.all_in_one_template.json`.

15 Output Controls

The output block controls what is written:

- `write_csv`: write EOS tables and summaries
- `write_json`: write the run summary
- `write_plots`: write figures
- `plot_formats`: choose `png`, `pdf`, or both

16 Practical Limitation

In the current implementation, sound-speed reconstruction is tied to the quarkyonic profile generation. Therefore:

- if you request a quarkyonic branch, v_s^2 is reconstructed
- if you do not want that branch, switch it off in `workflows`
- if you want the branch but do not want saved files, switch off `output.write_csv` and/or `output.write_plots`

17 Main Files To Use

For normal use, the main files are:

- `examples/qmm_blank_template.json`
- `examples/qmm_all_in_one_template.json`
- `notebooks/QMM_workflow.ipynb`
- `notebooks/baryquark_quarkyonic.ipynb`
- `examples/baryquark_quarkyonic_*.json`
- `scripts/compute_vdw_excluded_volume_only.py`
- `src/qmm/models.py`
- `src/qmm/config.py`
- `src/qmm/runner.py`